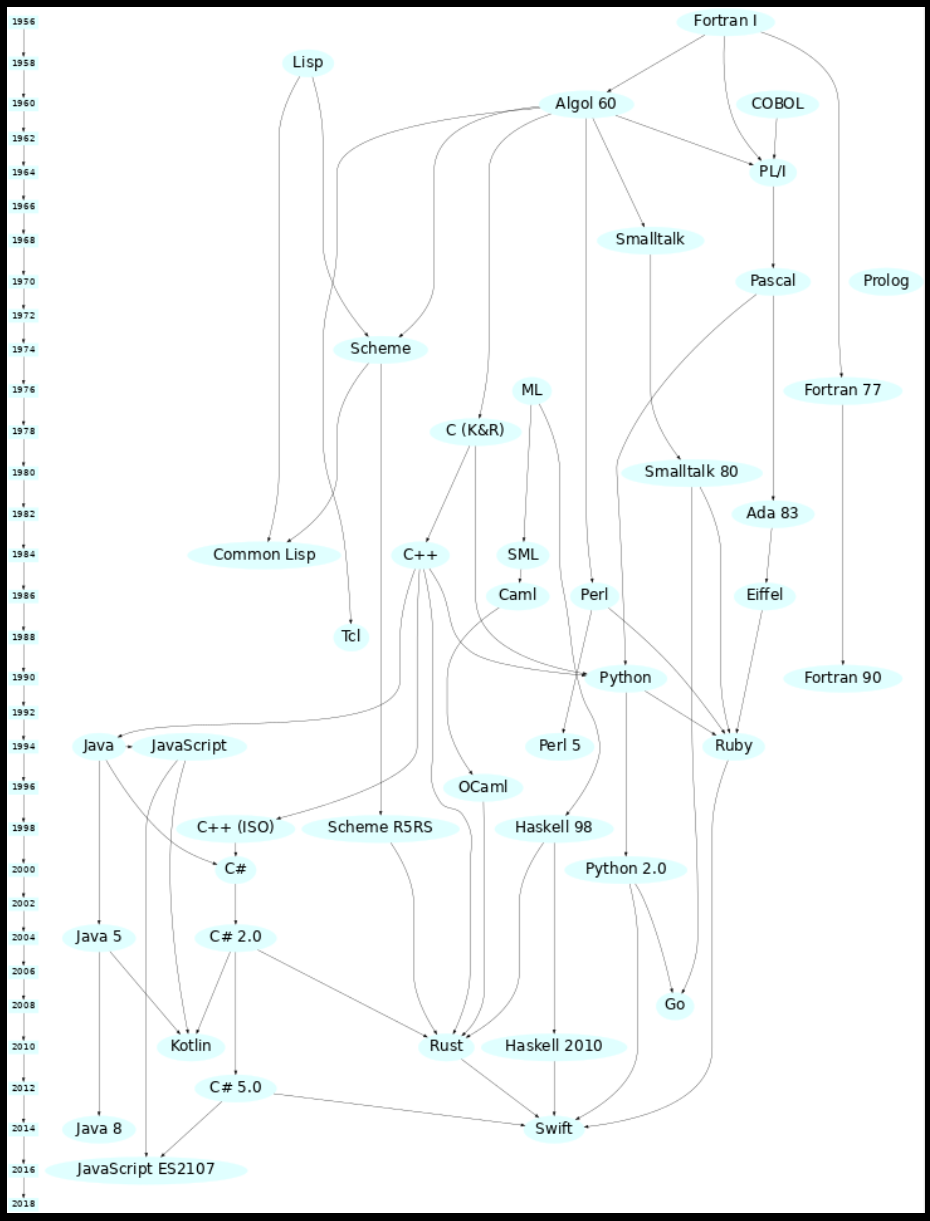
PDK 3.HAFTA Notları

Arda Kumartaşlıoğlu

29-09-2025

2 - Başlıca Programlama Dillerinin Evrimi

- 2.1 Zuse'nin Plankalkülü (Zuse's Plankalkül)
- 2.2 Minimal Donanım Programlama: Sözde Kodlar
- 2.3 IBM 704 ve Fortran
- 2.4 Fonksiyonel Programlama: Lisp
- 2.5 Sofistike Olmaya Doğru İlk Adım: ALGOL 60
- 2.6 İş Kayıtlarının Bilgisayar Ortamına Aktarılması: COBOL
- 2.7 Zaman paylaşımının başlangıcı: Basic
- 2.8 Herkes için her şey: PL/I
- 2.9 İki Erken Dinamik Dil: APL ve SNOBOL
- 2.10 Veri Soyutlamanın Başlangıcı: SIMULA 67
- 2.11 Ortogonal Tasarım: ALGOL 68
- 2.12 ALGOL'ların Bazı Erken Dönem Torunları
- 2.13 Mantığa Dayalı Programlama: Prolog
- 2.14 Tarihin En Büyük Tasarım Çalışması: Ada
- 2.15 Nesneye Yönelik Programlama: Smalltalk
- 2.16 Emir Kipi (Imperative) ve Nesneye Yönelik Özelliklerin Birleştirilmesi: C++
- 2.17 Emir Kipi (Imperative) Tabanlı Nesneye Yönelik Dil: Java
- 2.18 Scripting Dilleri
- 2.19 Amiral Gemisi .NET Dili: C#
- 2.20 İşaretleme(Markup)/Programlama Hibrit Dilleri



Ortak Dillerin Soy Ağacı

2.1 Zuse'nin Plankalkülü (Zuse's Plankalkül)

1945 yılında tasarlanmış, ancak 1972 yılına kadar yayınlanmamıştır.
Hiç uygulanmadı
Zuse, tek başına çalışarak, 1943 yılında doktora tezi için öneri olarak başlattığı Z4 için hesaplamaları ifade etmek üzere bir dil geliştirme çabasına girişti.
Bu dile Zuse'nin Plankalkülü (Zuse's Plankalkül), yani program hesaplaması adı verildi.

2.2 Minimal Donanım Programlama: Sözde Kodlar (Pseudocodes)

Makine kodu dezavantajları:

- Zayıf okunabilirlik
- Zayıf değiştirilebilirlik
- İfade kodlaması zahmetliydi
- Makine eksiklikleri - indeksleme veya kayan nokta yok

Gerçek programlama dili değiller ama program yazmayı kolaylaştırmak için tasarlandılar.

2.2.1 Short Code (1949)

John Mauchly tarafından geliştirildi (BINAC bilgisayarı için).
İlk yüksek seviyeli hesaplama sistemi denemelerindendir.
Programlar insan tarafından anlaşılır sembollerle yazılıyordu ama bilgisayarın anlayabilmesi için yorumlanması gerekiyordu (çok yavaştı).

2.2.2 Speedcoding (1954)

John Backus tarafından IBM 701 için geliştirildi.
Aritmetik işlemler ve matematik fonksiyonlar (sin, cos vb.) destekliyordu.
Koşullu ve koşulsuz dallanma (if, goto) içeriyordu.
Dizi elemanlarına otomatik erişim (register artışı) sağlıyordu.
Ama çok yavaştı — çünkü yorumlayıcıydı.
Kullanıcı programına sadece 700 kelime kadar yer kalıyordu.

2.3 IBM 704 ve Fortran

Fortran (Formula Translation) — İlk "gerçek" derlenebilir yüksek seviyeli dil.
Fortran 0 (1954) Tasarlandı ama uygulanmadı.
Fortran I (1957)

- IBM 704 için geliştirildi (kayan nokta destekliydi).
- Artık derleniyordu, yani kaynak kod makine koduna çevriliyordu.
- Programcılar ilk kez "insan diline yakın" yazabiliyordu.
- Özellikler:
  - DO döngüleri
  - Biçimlendirilmiş girdi/çıktı (I/O)
  - Alt programlar
  - Üç yönlü seçim (aritmetik IF)
  - Tür tanımlama yoktu

Fortran II (1958)

- Bağımsız derleme (modüler yapı) geldi.
- Hatalar düzeltildi.

Fortran IV (1960–62)

- IF–THEN–ELSE geldi.
- Karakter dizileri (string) desteklendi.
- Daha gelişmiş döngü kontrolü eklendi.

Fortran 90 (1990)

- Modüller
- Dinamik diziler
- Pointer'lar
- Özyineleme (recursion)
- CASE yapısı
- Tür denetimi

Fortran 95 / 2003 / 2008

- 1995: Küçük iyileştirmeler
- 2003: Nesne yönelimli programlama (OOP) ve C uyumu geldi
- 2008: Paralel diziler ve blok yapılar

Fortran'ın Önemi

- Derleyici çok iyi optimizasyon yapabiliyordu.
- Türler ve değişkenlerin bellek yerleri çalışmadan önce belirleniyordu.
- Bilgisayar kullanımını kökten değiştirdi.
- Hâlâ mühendislik ve bilim alanlarında kullanılır.
- Fortran = Bilimsel hesaplamanın temeli.

2.4 Lisp (1958)

- John McCarthy (MIT) tarafından geliştirildi.
- "LISt Processing" yani liste işleme dili.
- Yapay zekâ (AI) araştırmaları için tasarlandı.
- Veriler listeler halinde tutulur.
- Temeli lambda calculus (fonksiyonel matematik) fikrine dayanır.
- İki veri türü vardır:
- Atomlar (tekil semboller)
- Listeler (birden fazla atomdan oluşan yapı)

2.5 ALGOL 60 – İlk Sofistike (Gelişmiş) Dil

- O dönemde: FORTRAN yeni çıkmıştı, ama sadece belirli makinelerde çalışıyordu.
- Sorun: Evrensel, taşınabilir (makineye bağlı olmayan) bir dil yoktu.
- Çözüm: ALGOL 60 oluşturuldu — algoritmaları evrensel şekilde ifade edebilen bir dil.
- Yenilikleri:
  - Blok yapısı (yani değişkenlerin sadece belli bir kod parçasında geçerli olması)
  - Alt program (procedure) özyineleme (recursion)
  - Yerel değişkenler ve kapsam (scope)
  - BNF ile tanımlanmış söz dizimi (yani grameri matematiksel biçimde tanımlandı)

2.6 COBOL – İş Dünyası için Dil

- Nerede doğdu: ABD Savunma Bakanlığı'nın desteğiyle.
- Amacı: İş kayıtları ve veri tabanlı işlemler için kolay bir dil yaratmak.
- Özellikleri:
  - İngilizceye çok benzer söz dizimi ("ADD A TO B" gibi)
  - Uzun değişken isimleri, kayıt yapıları (records)
  - Veri tanımlama bölümü (Data Division)
  - Okunması kolay, teknik olmayan kişiler için bile anlaşılırdı.

2.7 BASIC – Öğrenmesi Kolay Dil

- Kim geliştirdi: Dartmouth Üniversitesi'nde Kemeny & Kurtz.
- Amacı: Bilim öğrencisi olmayan kişilerin de programlama öğrenebilmesi.
- Hedef: "Kullanıcı dostu, hızlı geri dönüş veren, ücretsiz" bir dil olması.
- Sonuç: Zaman paylaşımıyla (aynı anda birçok kişi bilgisayar kullanabiliyordu) popüler oldu.
- Günümüzdeki hali: Visual Basic.

2.8 PL/I – Herkese Uyan Tek Dil

Kim yaptı: IBM ve SHARE kullanıcı grubu.

Amaç: Hem bilimsel hesaplamaları (FORTRAN) hem iş uygulamalarını (COBOL) birleştirmek.

2.9 Dinamik Diller: APL ve SNOBOL

APL:
Matematiksel olarak çok güçlüydü, ama okunması zor bir dildi.
Diziler (arrays) ve matematiksel işlemler için mükemmeldi.
SNOBOL:
Metin ve desen (string pattern) işlemede güçlüydü.
Ama çok yavaştı ve artık nadir kullanılıyor.

2.10 SIMULA 67 – Nesne Yönelimli Dillerin Atası

Kim yaptı: Norveç'te Nygaard & Dahl.
Amacı: Sistem simülasyonları yapmak.

2.11 ALGOL 68 – Ortogonal Tasarım

ALGOL 60'ın geliştirilmiş hâliydi, ama tamamen onun devamı değildi.

2.12 Pascal – Öğretim Dili

Kim geliştirdi: Niklaus Wirth.
Amaç: Yapısal programlamayı öğretmek.
Basit, temiz ve öğretici bir dil.
Etkisi: 1970–1990 arasında programlama eğitiminin standart diliydi.

2.12.1 C – 1972

Kim geliştirdi: Dennis Ritchie, Bell Labs.
Amacı: Sistem programlama (işletim sistemleri gibi).
Özellikleri:
Güçlü operatörler, ama tip denetimi zayıf.
UNIX ile yayıldı.
Kullanımı: Sistem dili olmasına rağmen farklı alanlarda da kullanıldı.

2.13 Prolog – Mantığa Dayalı Programlama

Geliştiriciler: Comerauer ve Roussel, Kowalski katkısı.
Özellikleri:
Mantığa dayalı, prosedürel değil.
Verilen sorguların doğruluğunu kontrol eder (akıllı veri tabanı gibi).

2.14 Ada

Tarih: 1975–1978
Katılımcılar: Yüzlerce kişi, büyük bir proje.
Özellikleri:
Paketler, istisna işleme, eşzamanlılık (concurrency).
Program birimlerini detaylı kontrol edebilme.
Ada 95 ve 2005: OOP ve modern eşzamanlılık özellikleri eklendi.

2.15 Smalltalk – Nesne Yönelimli İlk Dil

Kim geliştirdi: Alan Kay, Adele Goldberg, Xerox PARC.
Özellikleri:

- Tam nesne yönelimli (OOP) uygulama.
- Veri soyutlama, kalıtım, dinamik bağlama.
- Grafik kullanıcı arayüzü tasarımında öncü oldu.

2.16 C++ – Emirsel + Nesne Yönelimli

Kim geliştirdi: Stroustrup, 1980.
Kaynak: C ve SIMULA 67
Özellikleri:
Hem prosedürel hem nesne yönelimli programlama destekler.
Büyük ve karmaşık bir dil, ama OOP ile popüler oldu.

2.17 Java – Emir Kipi Tabanlı OOP

Kim geliştirdi: Sun, 1990'lar.
Amaç: Gömülü cihazlar için basit, güvenli ve taşınabilir bir dil.
Özellikleri:
Sadece OOP, pointer yok.
Referans ve eşzamanlılık (thread) desteği.
Applet, GUI ve veri tabanı kütüphaneleri var.

2.18 Web için Scripting Dilleri

- Perl (1987): CGI programlama, UNIX yönetimi. Güçlü ama biraz riskli.
- JavaScript: Web tarayıcılarında HTML ile birlikte çalışır, yorumlanır.
- PHP: Web sunucusunda çalışır, form işleme ve veri tabanı erişimi için.
- Python: Nesne yönelimli, CGI ve web programlamada yaygın.
- Ruby: Saf OOP, yorumlanmış, operatörler kullanıcı tarafından özelleştirilebilir.
- Lua: Dinamik tip, CGI ve form işleme için kullanılır, basit ve genişletilebilir.

2.19 C# – .NET Dili

Platform: .NET, 2000.

Temel: C++, Java ve Delphi tabanlı.

Özellikler: Pointer, delegate, property, sınırlı dinamik tipleme, anonim türler.

2.20 Markup / Programlama Hibrit Dilleri

XSLT: XML belgelerini dönüştürmek için kullanılır, programlama yapıları içerir.

JSP (Java Server Pages): Dinamik web belgeleri için HTML ile birleşik programlama.

ÖZET

Bu bölümler, önemli programlama dillerinin tarihçesi, tasarım süreçleri ve katkılarını anlatıyor.

Her dilin ne amaçla geliştirildiği, neyi getirdiği ve eksikleri basit şekilde görülebiliyor.